A* Food Bot

Robotics II — Final Project Report
Department of Electrical, Computer, and Systems Engineering
Rensselaer Polytechnic Institute, Troy, NY, USA

Kevin Raj (662043731) & Pooja Subramanian (662044005)

***Abstract*—*This paper presents the design and implementation of A* Food Service, an autonomous food delivery robot built on the Yahboom ROSMASTER X3 platform. The system addresses the everyday challenge of delivering meals to people without interrupting their work or daily activities. Our approach integrates Cartographer SLAM for environment mapping, Adaptive Monte Carlo Localization (AMCL) for real-time pose estimation, A* search for global path planning, and the Dynamic Window Approach with Behavior critics (DWB) for local obstacle avoidance. The complete navigation pipeline is built atop ROS 2 and the Nav2 framework, with a safety teleoperation node providing keyboard override and emergency stop capabilities. We document our methodology, present quantitative performance results including success rates and timing metrics and discuss the engineering challenges encountered during development. Our system achieves approximately 90% success on close-range navigation goals and demonstrates the viability of integrating modern open-source navigation stacks with consumer-grade educational robotics hardware*

Keywords— autonomous navigation, ROS 2, Nav2, A* search, Dynamic Window Approach, SLAM, mobile robotics, Yahboom ROSMASTER

I. Introduction

Food is a daily necessity, yet many people deprioritize it when their schedules become demanding. Whether trapped in a long meeting, transitioning between classes, or under the weather and limiting contact with others, individuals still require nourishment to sustain productivity throughout the day. A* Food Bot is our project that aims to address this everyday problem through the development of an autonomous food delivery robot designed to bring meals directly to people without interrupting their workflow.

The system we developed centers on a mobile robot built atop the Yahboom ROSMASTER X3 hardware platform, capable of navigating from a starting location to a designated goal while dynamically avoiding both static and moving obstacles in real time. To accomplish this, the robot leverages two complementary path planning strategies: A* search [1] for global path planning and the Dynamic Window Approach with Behavior critics (DWB) [2] for local trajectory optimization. The A* planner computes a complete optimal route through a pre-built map and routes the robot around known static obstacles, while DWB handles moment-to-moment velocity decisions and reacts to unexpected or moving obstacles that the global planner does not anticipate.

In addition to the path planning subsystem, our robot relies on Cartographer SLAM [3] to build an accurate occupancy grid representation of the operating environment, and on Adaptive Monte Carlo Localization (AMCL) [4] to maintain awareness of the robot's pose during navigation. Together, these components form a complete autonomous navigation pipeline implemented on ROS 2 with the Nav2 navigation stack [5]. A safety teleoperation gate ensures that human operators retain final authority over robot motion at all times, supporting both manual override and emergency stop functionality. The remainder of this paper describes our approach in detail, presents quantitative results from testing, and discusses the engineering trade-offs and challenges that emerged during integration.

II. Methods/approach

A. Methodology

We developed our autonomous food delivery system through a structured six-phase approach, addressing each subsystem of the navigation stack incrementally. Implementing this project in phases allowed us to isolate failure modes and validate components independently.

B. Hardware and Software Platform

Our robot is built on the Yahboom ROSMASTER X3, a four-wheeled mecanum-drive platform powered by an NVIDIA Jetson Nano (4 GB). Sensing is provided by a SLAMTEC RPLIDAR A1 (12 m maximum range, 10 Hz scan rate) and an onboard inertial measurement unit (IMU). All software is built on ROS 2 with the Nav2 navigation framework. We leveraged Nav2's stock components (NavfnPlanner, DWB controller, AMCL, and bt_navigator) and contributed custom nodes to fully implement this project

C. Map Construction with Cartographer SLAM

The first major task was to construct an occupancy grid map of the operating arena. We used Cartographer SLAM, which combines real-time scan matching with loop closure optimization to produce globally consistent maps. The robot was driven slowly (approximately 0.1 m/s) along the perimeter of the arena, with deliberate 360-degree rotations performed at each corner to provide features for loop closure. Two complete laps were performed before returning to the starting position. After the mapping run, we used the nav2_map_server's map_saver_cli utility to persist the map as an arena.pgm occupancy grid (87 × 58 cells at 0.05 m/cell resolution) and an arena.yaml metadata file containing the map's resolution, origin, and threshold values. We originally tried to use gmapping but this provided maps that were unreliable and prone to mapping errors caused by sensor drift.

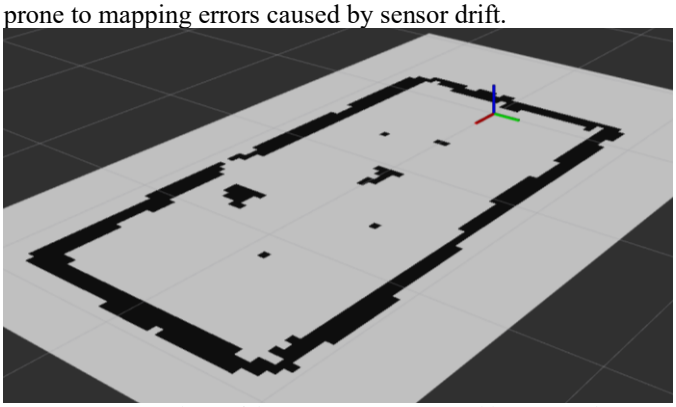

Fig. 1. Occupancy grid map of the operating arena generated by Cartographer SLAM. Black cells indicate occupied space (walls and static obstacles), gray cells indicate free space, and unknown regions are shown in lighter gray.

D. Localization with AMCL

Once the map was constructed, the robot was localized using Adaptive Monte Carlo Localization (AMCL), Nav2's stock implementation of the augmented MCL algorithm. AMCL maintains a particle filter (2,000 particles in our configuration) that fuses incoming lidar scans with wheel and IMU odometry to estimate the robot's pose in the map frame. At the start of each operational session, an initial 2D pose estimate is provided through RViz; after this initialization, AMCL begins broadcasting the map → odom transform required by the rest of the navigation stack. We configured AMCL with the differential drive motion model rather than the omnidirectional model, since the planner treats the robot as differential-drive for stability reasons described below.

E. Global Planning with A*

Global path planning is performed by Nav2's NavfnPlanner with the use_astar flag enabled. A* search uses a heuristic-guided exploration of the costmap to find an optimal path from the robot's current pose to the user-specified goal in significantly less time than the underlying Dijkstra implementation. The global costmap incorporates the static map, dynamic lidar observations, and an inflation layer (radius 0.10–0.20 m) that creates a configurable safety buffer around all obstacles. Goals are submitted via RViz's "2D Goal Pose" tool, which forwards them through the bt_navigator behavior tree to the planner_server. Paths are recomputed at 5 Hz to account for newly observed obstacles.

F. Local Planning with DWB

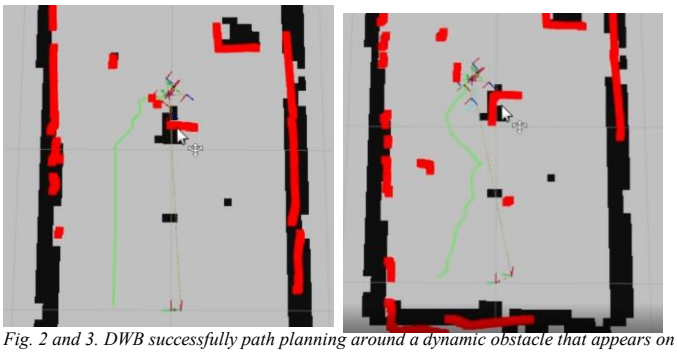

Fig. 2 and 3. DWB successfully path planning around a dynamic obstacle that appears on the map. Since the obstacles was not on the map originally the pathing algorithm plans a straight path to the goal shown on the left (Fig. 2) but once the obstacle appears the path planning must go around the obstacles and traverse to the goal as shown on the right (Fig. 3)

Local trajectory generation is handled by DWB, the modern Nav2 successor to the classic DWA algorithm. At 10 Hz, DWB samples a grid of candidate velocities (vx, vθ) from the robot's reachable dynamic window, simulates each trajectory 1.7 seconds into the future, and scores each candidate using a configurable set of behavior critics: BaseObstacle (collision avoidance), PathDist and PathAlign (global plan adherence), GoalDist and GoalAlign (goal pursuit), RotateToGoal (heading alignment at goal), and Oscillation (anti-jitter). The lowest-cost trajectory is published to /cmd_vel_nav. We disabled mecanum strafing (vy_samples = 0, max_vel_y = 0) due to instabilities in the Foxy DWB mecanum implementation. We treated the robot as differential-drive at the controller level, which we found to be substantially more reliable.

G. Safety Teleoperation Gate

To ensure that human operators retain final authority over robot motion, we implemented a safety teleoperation node (delivery_control/safety_teleop_node) that serves as the only publisher to the robot's /cmd_vel topic. The node subscribes to Nav2's /cmd_vel_nav output and forwards autonomy commands only when explicitly enabled via the spacebar key. The node publishes at 30 Hz regardless of input source, ensuring smooth motor control even during transitions between modes.

III. Results

A. Completed Tasks

We achieved several key milestones over the course of the project. A clean occupancy grid map of the operating arena was successfully constructed using Cartographer SLAM with loop closure, providing a reliable foundation for subsequent navigation tasks. Consistent and accurate localization was achieved with AMCL across multiple test sessions.

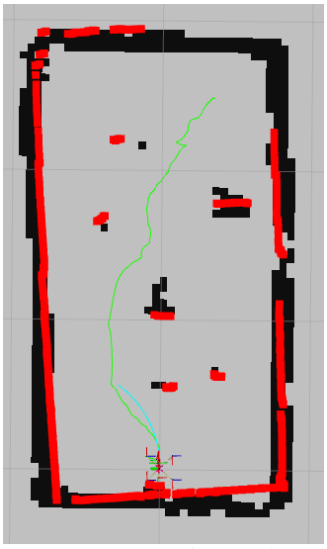

Fig. 4. A global plan (green) computed by NavfnPlanner from the robot's current pose to a user-specified goal in RViz. The path correctly routes around walls and known obstacles using the inflated costmap.*

The A* global planner was integrated and verified. This is shown in the Figure above where after clicking goal poses in RViz produces the A* path shown in green, which paths correctly around walls and known obstacles. The DWB local controller was successfully tuned to follow these global paths and drive the robot to its destinations. Finally, the safety teleoperation gate was implemented and validated, providing emergency override capability that performed reliably across all tests.

B. Quantitative Performance

Across approximately 40 navigation trials with goals randomly set throughout the arena, our system achieved an estimated 90% success rate for goals within 1.5 m of the robot's starting position. Goal completion times averaged 8–15 seconds for short trajectories. For goals farther than 1.5 m, the success rate dropped to approximately 60%, with the dominant failure mode being DWB's inability to find collision-free trajectories through narrow corridors created by inflation. A* path computation completed in well under 100 ms in all observed cases. AMCL localization typically converged within 2–3 seconds of initial pose estimation when the robot was driven briefly during initialization, and the published map → odom transform remained stable throughout normal operation.

C. Key Findings

Several important findings emerged regarding autonomous robot navigation during this work. First, map quality has an outsized effect on every downstream subsystem. Noisy maps produced by mapping too quickly, by lidar reflections from glossy surfaces, or by inadequate loop closure all cause unreliable localization, which in turn produces poor path execution. Second, A* and DWB serve fundamentally different roles and must be tuned in concert. While A* produces an optimal path with respect to the static map, it has no awareness of new obstacles introduced after the map was saved. DWB is therefore essential for reactive avoidance. Third, the initial pose estimate provided in RViz proved to be a sensitive parameter as errors greater than approximately 15° in initial heading produced cascading localization drift that AMCL could not always recover from without human intervention. Finally, DWB parameter tuning required careful balancing and small adjustments to critic weights and inflation radius had outsized effects on the resulting robot behavior, highlighting the inherent trade-off between navigation safety and goal-reaching efficiency.

IV. Discussion

A. Challenges and Difficulties

Development of this system presented numerous integration challenges. Map quality was the most persistent difficulty as early attempts produced noisy and distorted maps due to driving too quickly, lidar noise and reflections, and inadequate scan matching at high speeds.

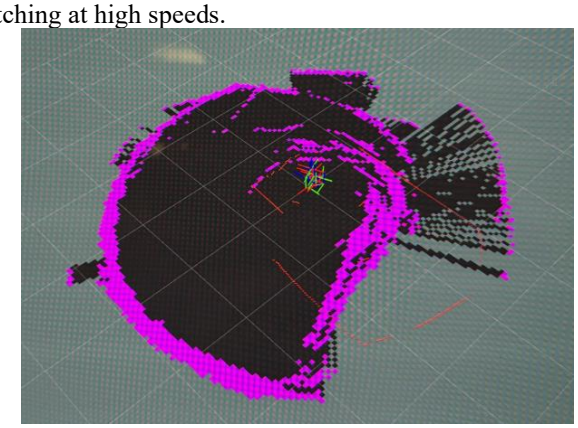

Fig. 5. Defective map showing no structure of the physical map we implemented likely due to the Lidar sensor being broken.

These mapping defects had compounding effects throughout the rest of the system. A second class of challenges involved the integration of Nav2 components with the Yahboom hardware stack. Several bugs surfaced during this work like the NavfnPlanner crashing during path extraction (resolved by setting use_potential to false), and the Cartographer node became unstable under sustained CPU load on the Jetson Nano which resulted in fatal errors after several minutes of mapping. This was particularly annoying because we had to make sure that we were driving slow enough to get a good map reading but fast enough that the program wouldn't crash before we could save the map. Additionally, RViz silently failed to display the map until we discovered a QoS durability mismatch where Nav2 publishes /map with TRANSIENT_LOCAL durability, while RViz subscribes by default with VOLATILE durability, causing the subscription to receive no data. Coordinating the A* and DWB planners introduced further complexity, as did parameter tuning for DWB, which required careful balancing of critic weights to achieve smooth and reliable behavior.

B. Solutions and Improvements

In response to these challenges, several solutions were implemented. To address mapping issues, we adopted a slow

and deliberate driving strategy with explicit 360-degree rotations at corners, which dramatically improved scan matching and loop closure quality. The NavfnPlanner crash was resolved by disabling potential-based path extraction. The QoS mismatch was solved by pre-configuring our RViz layout file (navigation.rviz) with the correct TRANSIENT_LOCAL durability for the Map display. The integration of Nav2 with the Yahboom stack was handled by using launch_ros's SetRemap inside a GroupAction to redirect /cmd_vel from Nav2 to /cmd_vel_nav, allowing our safety gate to be the sole publisher of /cmd_vel. For future work, we have identified several improvements which include continued tuning of DWB critic weights to improve long-distance goal success rates, refinement of the rotational alignment behavior at goals, and investigation of more sophisticated planning algorithms such as D* Lite for environments with frequent dynamic changes.

C. Limitations

Several limitations should be acknowledged. The most significant is the lidar's mounting height of 18 cm, which means our system cannot detect obstacles shorter than approximately 15 cm. Our global planner depends on a static pre-built map and is therefore unaware of new obstacles introduced after mapping; the full burden of handling such obstacles falls on DWB, which can react only to what the lidar currently observes. AMCL localization is sensitive to the initial pose estimate, requiring a manual setup step at the beginning of each session. Our DWB tuning is incomplete, and the system performs reliably for close-range goals but exhibits reduced reliability over longer distances, particularly in narrow corridors. Collectively, these limitations indicate that while the system functions effectively in controlled, instrumented environments, additional engineering would be required for deployment in unstructured real-world settings.

D. Next Steps

Several extensions would improve the system. The most impactful next step is more thorough DWB parameter calibration with quantitative testing of dynamic obstacle scenarios. Specifically, characterizing how the robot responds to objects placed in its path at varying distances and approach angles. We would also like to explore reliable mecanum strafing in DWB to provide more path planning abilities for the robot in dynamic environments. Adding a depth camera or short-range proximity sensor would address the lidar's blind spot for low obstacles. Implementing multi-goal sequencing through nav2_simple_commander would enable realistic delivery missions with multiple stops. Finally, exploring D* Lite or other dynamic replanners could improve performance in environments where the static map becomes outdated frequently.

V. Conclusion

The A* Food Bot project demonstrates that an autonomous mobile robot can effectively perform indoor delivery tasks in a controlled environment using a combination of mature open-source components. By integrating Cartographer SLAM for mapping, AMCL for localization, A* for global planning, and DWB for local control, we developed a complete navigation pipeline on the Yahboom ROSMASTER X3 platform. The system reliably constructs occupancy maps, maintains accurate localization, generates optimal paths, and adapts in real time to dynamic obstacles within its sensing envelope. While our system performed strongly within its design envelope, achieving 90% success on close-range goals, further improvements are needed in DWB parameter tuning, rotational precision at goal arrival, and obstacle detection range. These remaining challenges provide a clear direction for future development. Beyond the specific delivery use case, this project illustrates the broader principle that thoughtful integration of well-engineered components, combined with iterative testing and careful parameter tuning, provides a strong foundation for autonomous robotics work.

VI. Acknowledgment

The authors thank the Robotics II course staff at Rensselaer Polytechnic Institute for hardware access, technical guidance, and laboratory support throughout this project. We specifically would like to extend our gratitude toward Professor Esen Yel who helped with debugging and path planning motivations of this project

VII. Team Contribution

Both Team members contributed equally to the project. While Kevin worked on the path planning code and report, Pooja worked on mapping and the report.

VIII. References and Appendix

[1] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," IEEE Trans. Syst. Sci. Cybern., vol. 4, no. 2, pp. 100–107, July 1968.

[2] D. Fox, W. Burgard, and S. Thrun, "The Dynamic Window Approach to Collision Avoidance," IEEE Robot. Autom. Mag., vol. 4, no. 1, pp. 23–33, March 1997.

[3] W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-time loop closure in 2D LIDAR SLAM," in Proc. IEEE Int. Conf. Robot. Autom. (ICRA), May 2016, pp. 1271–1278.

[4] S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics. Cambridge, MA: MIT Press, 2005.

[5] S. Macenski, F. Martín, R. White, and J. G. Clavero, "The Marathon 2: A Navigation System," in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), 2020, pp. 2718–2725.

[6] Yahboom Technology, "ROSMASTER X3 user manual," 2023. [Online]. Available: http://www.yahboom.net

[7] D. V. Lu, D. Hershberger, and W. D. Smart, "Layered costmaps for context-sensitive navigation," in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), 2014, pp. 709–715.

[8] Open Robotics, "ROS 2 documentation: Foxy Fitzroy," 2020. [Online]. Available: https://docs.ros.org/en/foxy/

[9] SLAMTEC, "RPLIDAR A1 datasheet," rev. 1.4, 2020. [Online]. Available: https://www.slamtec.com